

Day 11 — DNS in Depth

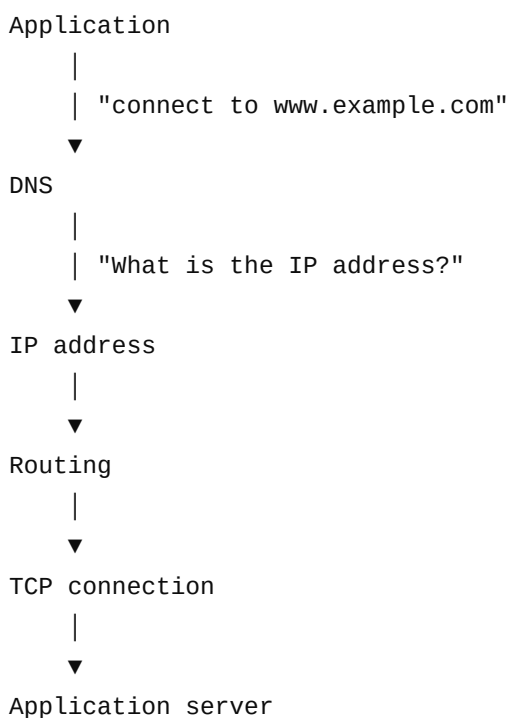
Hostname → DNS Resolver → DNS Servers → IP Address → Application Connection

Today we'll learn **DNS from the packet-flow perspective**, not just definitions.

You already learned:

- Layer 2 — MAC, ARP, Switch
- Layer 3 — IP, routing
- Layer 4 — TCP/UDP
- ICMP, ping, traceroute
- Linux routing commands

Now we connect them:



Day 11 Goals

By the end of today, you should understand:

1. What DNS is
2. Why DNS is required
3. Domain name vs IP address
4. DNS resolver
5. Recursive DNS server
6. Root DNS server
7. TLD server
8. Authoritative DNS server
9. DNS lookup flow
10. DNS cache
11. DNS records

12. UDP vs TCP for DNS
 13. DNS port 53
 14. Linux DNS configuration
 15. How to troubleshoot DNS
 16. What happens when you type a website name
 17. DNS interview questions
-

1. What is DNS?

DNS = Domain Name System

DNS translates:

Human-friendly name
↓
IP address

Example:

www.example.com
↓
DNS lookup
↓
93.184.216.34

Why?

Humans prefer:

www.example.com

Computers communicate using addresses such as:

93.184.216.34

2. Why Do We Need DNS?

Imagine there was no DNS.

You would have to remember:

142.250.x.x
172.217.x.x
...

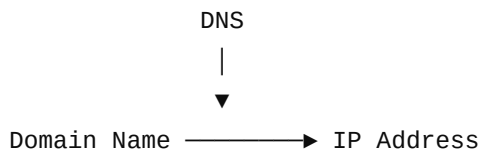
for every service.

Instead:

google.com
youtube.com
github.com

can be translated into IP addresses.

So:



3. DNS Is Not Routing

This distinction is important.

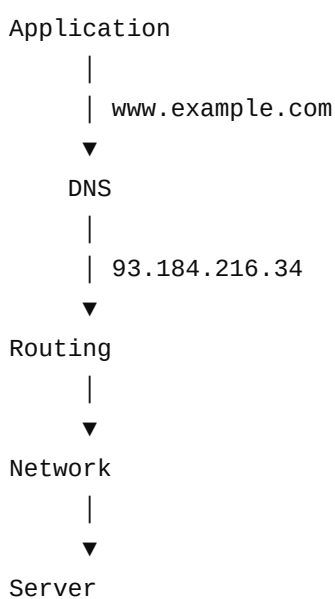
DNS answers:

"What IP address belongs to this hostname?"

Routing answers:

"How do I reach that IP address?"

Example:



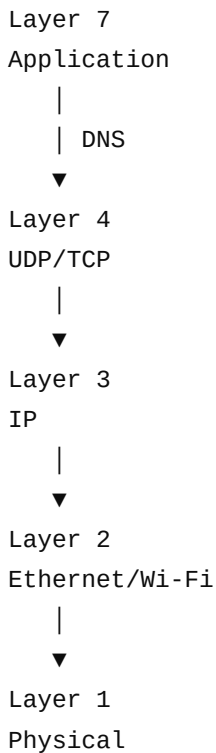
So:

DNS → Name → IP
Routing → IP → Path

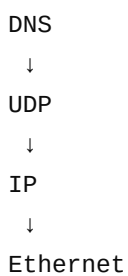
4. DNS and the OSI Layers

DNS is an **application-layer** protocol.

Conceptually:



For normal DNS queries, you'll commonly see:



DNS can also use TCP.

We'll discuss why later.

5. DNS Uses Port 53

The traditional DNS service uses:

UDP port 53

TCP port 53

Important interview point:

DNS commonly uses UDP port 53 for ordinary queries, but TCP port 53 is also used in several cases.

So don't say:

"DNS always uses UDP."

That is incorrect.

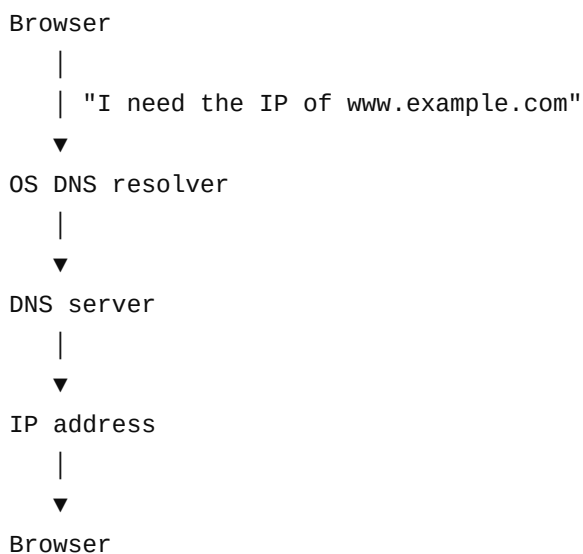
6. Basic DNS Flow

Suppose you type:

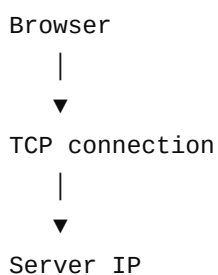
www.example.com

into a browser.

Conceptually:



Then:



7. Important: Browser Does Not Usually Ask Root DNS Directly

A common beginner misconception is:

```
Browser
  ↓
Root DNS
  ↓
TLD
  ↓
Authoritative
```

Usually, that's not what happens from your PC.

More accurately:

```
Your PC
  |
  ▼
Configured DNS Resolver
  |
  ├── Root DNS
  ├── TLD DNS
  └── Authoritative DNS
```

The **recursive resolver** performs the recursive lookup work.

8. Who Are the DNS Players?

There are four important concepts:

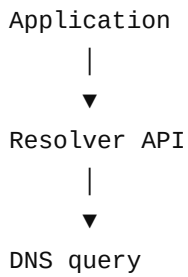
1. DNS Client / Stub Resolver
2. Recursive Resolver
3. Root DNS Server
4. TLD DNS Server
5. Authoritative DNS Server

Let's understand them.

9. DNS Client / Stub Resolver

Your Linux PC has a resolver component that receives requests from applications.

For example:



Applications may use system resolver facilities rather than implementing the entire DNS protocol themselves.

For example, Linux programs can use APIs such as:

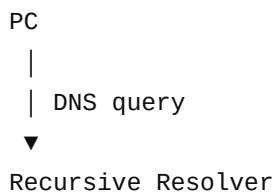
`getaddrinfo()`

We'll look at this API later.

10. Recursive DNS Resolver

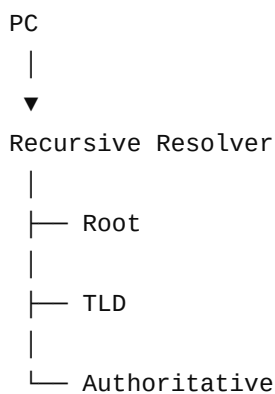
Your PC usually talks to a **recursive resolver**.

For example:



The recursive resolver does the work of finding the answer if it isn't already cached.

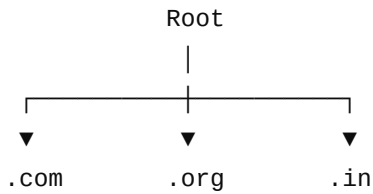
Conceptually:



11. Root DNS Servers

At the top of the DNS hierarchy are the **root DNS servers**.

Conceptually:



The root doesn't normally provide the final IP address for:

`www.example.com`

Instead, it can tell the resolver:

"For `.com`, ask the `.com` TLD servers."

12. TLD

TLD = Top-Level Domain

Examples:

`.com`
`.org`
`.net`
`.in`
`.uk`

For:

`www.example.com`

the TLD is:

`.com`

The TLD DNS server can direct the resolver toward the authoritative DNS servers for:

`example.com`

13. Authoritative DNS Server

The authoritative DNS server contains the authoritative DNS information for a domain.

For:

example.com

the authoritative server knows records such as:

www.example.com → IP address

Conceptually:

```
Root
  ↓
.com TLD
  ↓
example.com authoritative DNS
  ↓
www.example.com → IP
```

14. Full DNS Lookup

Let's follow:

www.example.com

Step 1

Your PC asks the recursive resolver:

"What is the IP of www.example.com?"

```
PC
|
▼
Recursive Resolver
```

Step 2

If not cached, resolver asks root:

```
Resolver
|
▼
Root
```

Root says conceptually:

"I don't know the final IP,
but ask the .com TLD servers."

Step 3

Resolver
|
▼
.com TLD

TLD says:

"Ask example.com's authoritative DNS server."

Step 4

Resolver
|
▼
Authoritative DNS

Authoritative server returns:

www.example.com
↓
93.184.216.34

Step 5

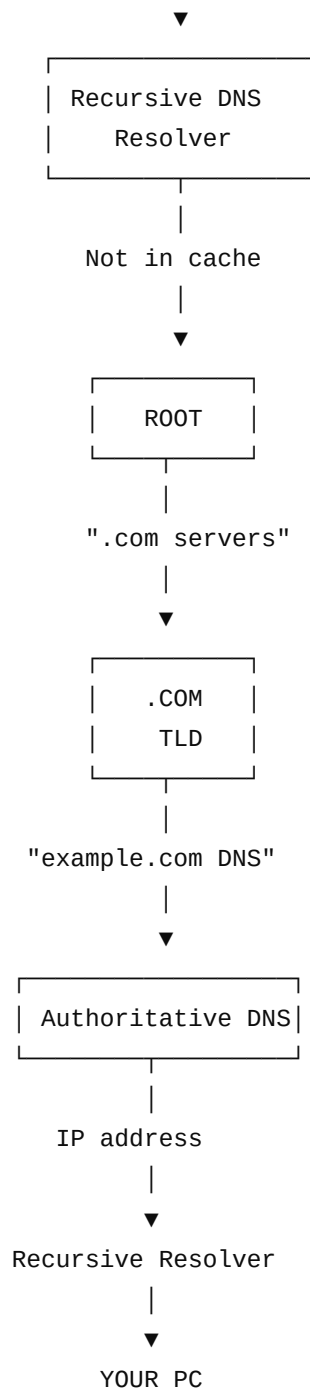
Resolver returns answer:

Recursive Resolver
|
▼
Your PC

15. Visualize the Complete DNS Lookup

YOUR PC
|
|
| DNS Query





16. Recursive vs Iterative

This is a common interview topic.

Client → Recursive Resolver

The client essentially asks:

"Please find the answer for me."

That's a **recursive request**.

PC

| "Find www.example.com"



Resolver

The resolver does the work.

Resolver → Root/TLD/Authoritative

The resolver may perform iterative queries:

Resolver



Root



TLD



Authoritative

The servers progressively direct the resolver to the next place to ask.

17. DNS Cache

Imagine every DNS request always required:

Root



TLD



Authoritative

That would be inefficient.

DNS therefore uses caching.

PC



Recursive Resolver



└─ Cache HIT → answer immediately



└─ Cache MISS → perform lookup

18. Cache Hit

Suppose the resolver already knows:

www.example.com
↓
93.184.216.34

Then:

PC
|
▼
Resolver
|
| Cache HIT
▼
IP address

No need to contact root/TLD/authoritative servers again.

19. Cache Miss

If the answer isn't available:

PC
|
▼
Resolver
|
| Cache MISS
▼
Root
|
▼
TLD
|
▼
Authoritative
|
▼
IP

Then the resolver caches the result.

20. TTL in DNS

Don't confuse **DNS TTL** with **IP packet TTL**.

We now have two different TTL concepts.

IP TTL

Used to prevent routing loops:

```
IP packet
TTL 64
↓
Router
TTL 63
```

DNS TTL

Controls how long a DNS record may be cached.

Example:

```
www.example.com
IP = 93.184.216.34
TTL = 300 seconds
```

Meaning a resolver can generally cache that record for the specified TTL before it needs to refresh it.

Interview trap:

```
IP TTL ≠ DNS TTL
```

21. DNS Records

You should know these important DNS record types:

Record	Purpose
A	Hostname → IPv4
AAAA	Hostname → IPv6
CNAME	Alias → canonical hostname
MX	Mail server
NS	Authoritative name server
TXT	Text/metadata

Record

Purpose

PTR

Reverse DNS

22. A Record

Example:

www.example.com

↓

93.184.216.34

This is an:

A record

because it maps:

Hostname → IPv4

23. AAAA Record

For IPv6:

www.example.com

↓

2001:db8::10

This is:

AAAA

So:

A → IPv4

AAAA → IPv6

Very common interview question.

24. CNAME

Suppose:

www.example.com
↓
server.example.com
↓
IP address

www.example.com can be an alias using a:

CNAME

Conceptually:

www
|
▼
server.example.com
|
▼
A/AAAA record
|
▼
IP

25. MX Record

Used for mail delivery.

Example:

example.com
|
▼
MX
|
▼
mail.example.com

It tells mail systems which servers handle email for a domain.

26. NS Record

NS = Name Server

It identifies authoritative DNS servers for a domain.

Conceptually:

example.com



NS



ns1.example-dns.com

27. PTR Record

PTR is used for:

Reverse DNS

Normal DNS:

Name → IP

Reverse DNS:

IP → Name

Example:

8.8.8.8



hostname

28. Forward vs Reverse DNS

FORWARD

google.com



IP

REVERSE

IP



hostname

Interview question:

Which DNS record is commonly used for reverse lookup?

Answer:

PTR

29. DNS Query Using Linux

You can inspect DNS using:

```
nslookup example.com
```

or preferably:

```
dig example.com
```

if `dig` is installed.

30. dig

Run:

```
dig example.com
```

You might see:

```
QUESTION SECTION:
```

```
example.com.      IN      A
```

```
ANSWER SECTION:
```

```
example.com.      300     IN      A      93.184.216.34
```

Interpret:

```
example.com
```

```
↓
```

```
A record
```

```
↓
```

```
93.184.216.34
```

And:

300

is the DNS record TTL in seconds.

31. Ask Specifically for A

dig example.com A

Means:

Give me the IPv4 address record.

32. Ask for AAAA

dig example.com AAAA

Means:

Give me the IPv6 address record.

33. Ask for MX

dig example.com MX

Means:

Show mail exchanger records.

34. Ask for NS

dig example.com NS

Means:

Show name-server records.

35. Reverse DNS

You can use:

```
dig -x 8.8.8.8
```

Conceptually:

```
8.8.8.8
  ↓
PTR lookup
  ↓
hostname
```

36. DNS and Your Browser

Now let's connect DNS with the complete networking flow.

Suppose you enter:

```
https://example.com
```

The simplified process is:

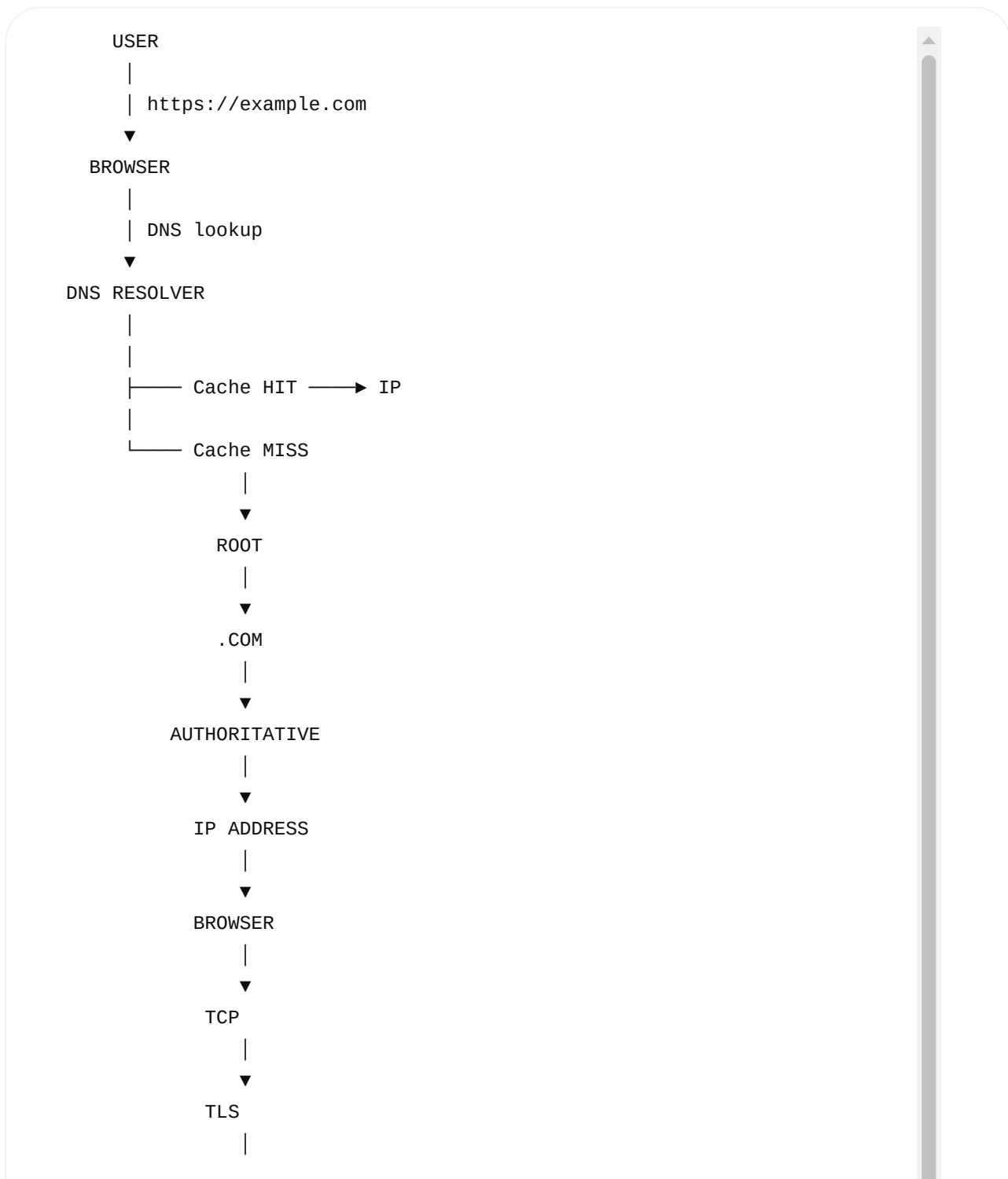
```
Browser
  |
  | Need IP
  ▼
DNS
  |
  | IP address
  ▼
TCP
  |
  | connection
  ▼
TLS
  |
  | secure session
```



So DNS happens **before the browser can normally establish the connection to the destination IP**.

There can be caching and connection reuse, so real browsers can optimize this process.

37. Complete Visualization





38. What Happens at the Network Layer During DNS?

This connects to everything you've learned.

Suppose DNS resolver is:

8.8.8.8

Your PC needs to send a DNS query.

The networking stack does:

```
graph TD; A[DNS Query] --> B[UDP]; B --> C[Destination Port 53]; C --> D[IP]; D --> E[Routing lookup]; E --> F[Default Gateway]; F --> G[ARP / Neighbor lookup]; G --> H[Ethernet]; H --> I[NIC];
```

So DNS itself is application layer, but its packet travels through:

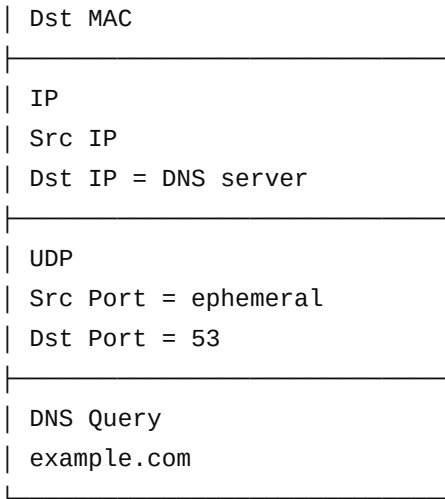
Layer 7 → Layer 4 → Layer 3 → Layer 2 → Layer 1

39. DNS Query Packet

Conceptually:

Ethernet Frame

```
graph LR; A[Src MAC];
```



This is an excellent interview-level connection.

40. Why Does DNS Usually Use UDP?

UDP is:

- Connectionless
- Low overhead
- Fast

A typical DNS query can fit into a relatively small packet.

Therefore:

```
graph TD
    DNS[DNS] --> UDP[UDP]
    UDP --> Port[Port 53]
```

The diagram shows a vertical flow of information. It starts with 'DNS' at the top, followed by a downward arrow, then 'UDP', followed by another downward arrow, and finally 'Port 53' at the bottom.

is common.

41. When Does DNS Use TCP?

DNS can use TCP when needed.

Important cases include:

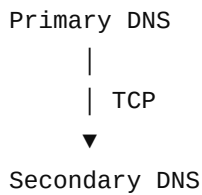
1. Large DNS responses

Older DNS behavior commonly involved UDP size limitations, and larger responses can require TCP fallback.

2. Zone transfers

DNS zone transfers commonly use TCP.

For example:



3. Other protocol-specific requirements

Modern DNS has additional mechanisms for handling larger UDP responses, so don't simplify this to:

"Large response always means TCP."

Better interview answer:

"DNS commonly uses UDP 53, but TCP 53 is also used when required, including zone transfers and certain response/transport conditions."

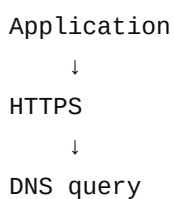
42. DNS Over HTTPS

Modern systems can also use:

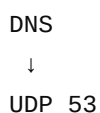
DoH

DNS over HTTPS

Conceptually:



Instead of traditional:



DoH commonly uses:

HTTPS
TCP/TLS
port 443

For your interview, know the concept; don't worry about deep DoH implementation yet.

43. DNS Over TLS

Another modern mechanism:

DoT

DNS over TLS

Conceptually:

```
DNS
↓
TLS
↓
TCP
↓
Port 853
```

Know the distinction:

```
Traditional DNS → UDP/TCP 53
DoT             → TCP 853
DoH             → HTTPS, commonly 443
```

44. Linux DNS Configuration

On Linux, DNS configuration depends on the distribution and resolver setup.

You may encounter:

```
/etc/resolv.conf
```

For example:

```
nameserver 192.168.1.1
```

This tells the resolver which DNS server to use.

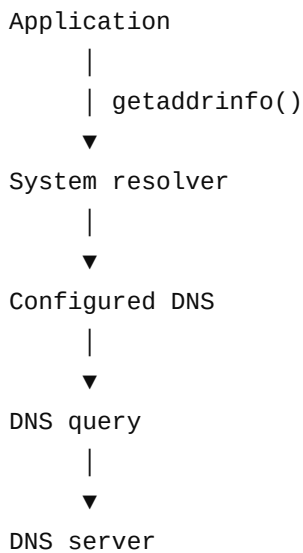
But on systems using `systemd-resolved`, `/etc/resolv.conf` may point to a local stub resolver rather than directly listing the upstream DNS server.

You can inspect:

```
resolvectl status
```

45. Linux DNS Flow

A simplified Linux flow:



Important API:

```
getaddrinfo()
```

This is commonly used by applications to resolve hostnames and obtain address information.

46. getaddrinfo()

A typical application might do:

```
struct addrinfo hints = {0};
struct addrinfo *result;

getaddrinfo("example.com", "443", &hints, &result);
```

Conceptually:

```
"example.com"
+
"443"
```



```
↓
▼
getaddrinfo()
↓
▼
DNS / local resolver mechanisms
↓
▼
IP address + address information
```

Then the application can use the result with:

```
socket()
connect()
```

47. Important Connection to Your Previous `connect()` Question

You previously asked whether:

```
connect() directly connects to the transport layer.
```

Now consider:

```
connect(sock, ...);
```

If the application starts with a hostname, it may first need:

```
Hostname
↓
DNS resolution
↓
IP address
↓
connect()
↓
TCP
↓
Routing
↓
ARP
↓
Ethernet
```

But don't assume `connect()` itself always performs DNS resolution.

Usually the application/resolver first obtains the address, often using:

```
getaddrinfo()
```

and then calls:

```
connect()
```

with the resulting address.

48. DNS Failure vs Routing Failure

This distinction is very important in troubleshooting.

Case 1

```
ping 8.8.8.8
```

works.

But:

```
ping google.com
```

fails.

Possible:

```
DNS problem
```

Case 2

```
ping 8.8.8.8
```

fails.

Then DNS isn't necessarily the first thing to blame.

Think:

Interface

↓

IP

↓

Route

↓

Gateway

↓

Network

49. DNS Troubleshooting

Suppose:

```
ping 8.8.8.8
```

works.

But:

```
ping example.com
```

fails.

Use:

```
dig example.com
```

or:

```
nslookup example.com
```

Check:

```
resolvectl status
```

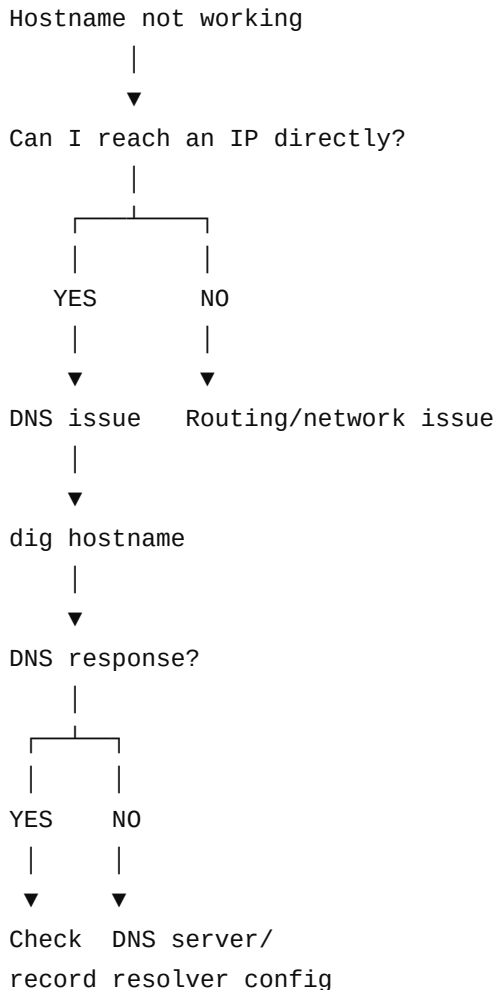
Then inspect:

```
cat /etc/resolv.conf
```

depending on your system's resolver configuration.

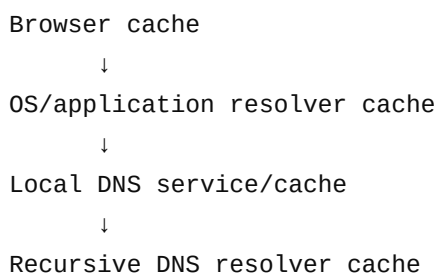
50. DNS Troubleshooting Flow

Memorize this:



51. DNS Cache on Your PC

There may be several layers of caching:



Therefore, changing a DNS record doesn't necessarily mean every client immediately sees the new address.

DNS TTL affects caching.

52. Example: Website IP Changes

Suppose:

example.com

↓

10.1.1.10

Later administrator changes it:

example.com

↓

10.1.1.20

A resolver may still have:

10.1.1.10

cached until the record's TTL expires.

After refresh:

10.1.1.20

can be returned.

53. DNS Does Not Establish TCP

Very important.

DNS gives you:

Hostname

↓

IP

It doesn't establish:

TCP connection

For example:

DNS

↓

93.184.216.34

Then:

connect()

↓

TCP SYN

↓

SYN-ACK

↓

ACK

Then the application protocol can proceed.

54. Full Website Flow — Everything We've Learned

This is the big picture.

USER

|

| https://example.com

▼

BROWSER

|

| "What is example.com's IP?"

▼

DNS

|

| — Resolver

|

| — Cache?

|

| — Root → TLD → Authoritative

|

▼

IP ADDRESS

|

▼

Routing Table

|

▼

Gateway

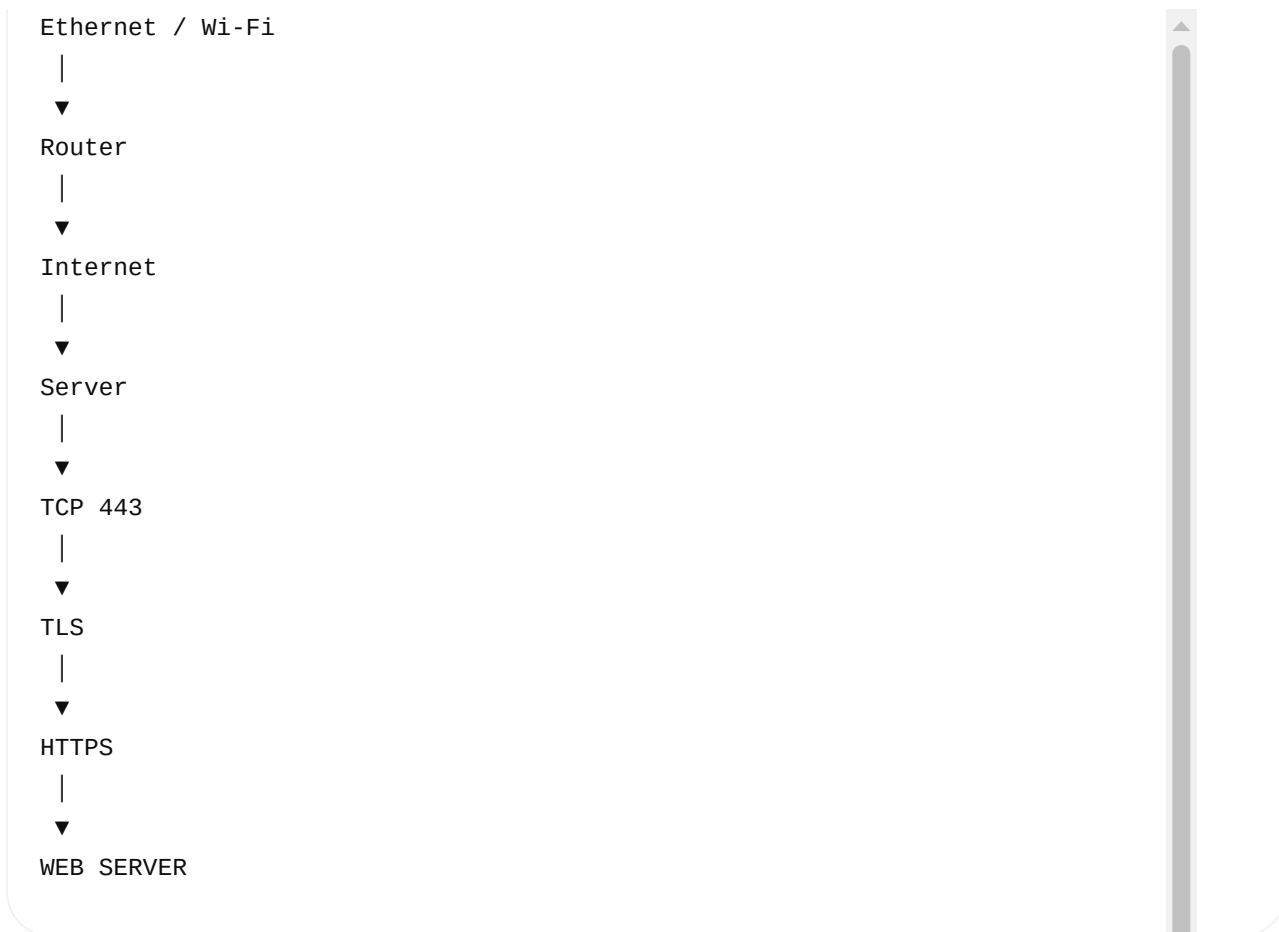
|

▼

ARP / Neighbor

|

▼



★ Day 11 Interview Questions

Try answering these yourself first.

Basic

1. What is DNS?
2. Why do we need DNS?
3. What layer does DNS belong to?
4. What port does DNS use?
5. Does DNS use TCP or UDP?
6. What is a DNS resolver?
7. What is a recursive DNS server?
8. What is a root DNS server?
9. What is a TLD server?
10. What is an authoritative DNS server?

DNS Records

11. What is an A record?
12. What is an AAAA record?
13. What is a CNAME record?
14. What is an MX record?
15. What is an NS record?
16. What is a PTR record?
17. What is forward DNS?

18. What is reverse DNS?

Practical

19. What does `dig example.com` do?

20. What does `dig example.com A` do?

21. What does `dig -x 8.8.8.8` do?

22. What does `resolvectl status` show?

23. What is `/etc/resolv.conf`?

24. How do you troubleshoot DNS failure?

25. Why might `ping 8.8.8.8` work while `ping google.com` fails?

Deep interview questions

26. Explain the complete DNS lookup process.

27. Explain recursive vs iterative DNS queries.

28. What is DNS caching?

29. What is DNS TTL?

30. Difference between DNS TTL and IP TTL?

31. Why does DNS commonly use UDP?

32. When can DNS use TCP?

33. What is DoH?

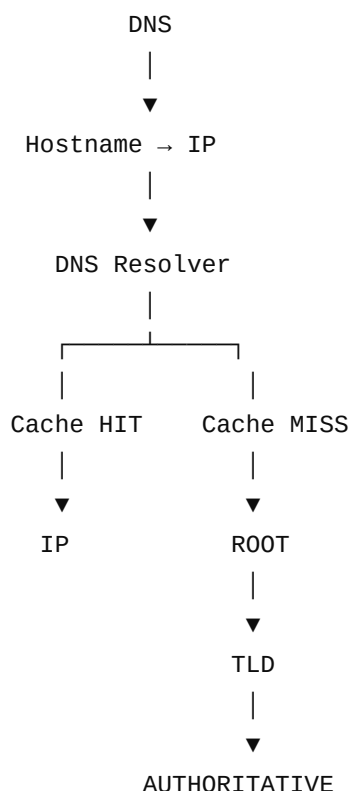
34. What is DoT?

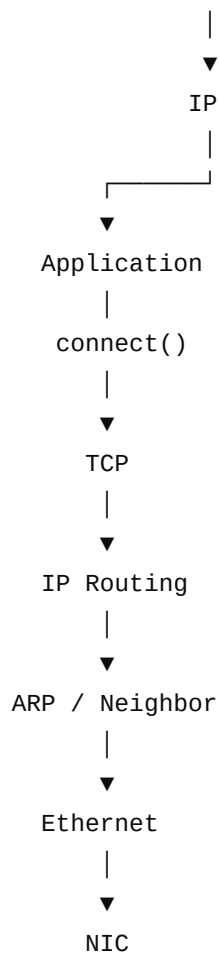
35. Does DNS establish a TCP connection to the web server?



Day 11 — Must Remember

Keep this diagram for revision:





The three statements you should be able to say confidently:

DNS converts a hostname into an IP address.

Routing determines how to reach that IP address.

TCP/UDP and the application protocol then provide the actual communication.

Day 11 Hands-On on Ubuntu

Run these commands and observe the output:

```
ip addr
ip route
resolvectl status
cat /etc/resolv.conf
```

Then:

```
ping google.com
```

and:

```
ping 8.8.8.8
```

Then:

```
dig google.com  
dig google.com A  
dig google.com AAAA  
dig google.com MX  
dig -x 8.8.8.8
```

Finally compare:

```
ip route get 8.8.8.8
```

with:

```
ip neigh
```

This will tie **Day 7** → **Day 8** → **Day 9** → **Day 10** → **Day 11** together:

```
MAC/ARP  
↓  
IP/Subnet  
↓  
Routing  
↓  
ICMP  
↓  
DNS  
↓  
TCP  
↓  
Application
```

Day 11 complete. Next: Day 12 — HTTP/HTTPS, TCP connection → TLS handshake → HTTP request/response, and what actually happens when you open a website.